

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

И.Д. Свеженин

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №6

СУБД и использование сетевых сервисов

по курсу: Кроссплатформенное программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1. Задание	2
2. Ход работы.....	3
3. Результат работы программы.....	5
4. Вывод.....	8
5. Листинг с кодом программы.....	9

1. Задание

Задание заключалось в проектировании и разработке мобильного приложения под Android на Kotlin, включая два основных блока. Первый блок работа с базой данных требовал создания базы данных для определенного класса с использованием библиотеки Room, реализации добавления данных через отдельное Activity, вывода данных из базы в RecyclerView с использованием фрагментов, обработки долгого нажатия на элементы списка, создания диалогового окна с опциями просмотра удаления и обновления, обработки действий диалога включая подтверждение операций, а также реализации обновления и удаления данных через отдельные Activity. Второй блок работа с JSON включал загрузку JSON данных из интернета с использованием HttpURLConnection или Retrofit, парсинг полученных данных и выполнение операций с JSON в отдельном потоке Thread для обеспечения отзывчивости интерфейса приложения.

2. Ход работы

Начальный этап включал настройку проекта и подготовку инфраструктуры базы данных. В Android Studio создан проект на Kotlin, подключены необходимые зависимости через Gradle, включая библиотеку Room для работы с базой данных, Retrofit и Gson для сетевых запросов и парсинга JSON, а также Coroutines для асинхронных операций. Определена предметная область приложения для работы с информацией о гонщиках Формулы 1, создана структура базы данных с использованием Room, включая Entity класс Driver с полями для хранения информации о гонщиках, интерфейс DriverDao с методами для выполнения операций CRUD, и класс AppDatabase для управления базой данных.

Второй этап был посвящен разработке пользовательского интерфейса и реализации отображения данных. Создана главная Activity с нижней навигацией и плавающими кнопками для добавления и обновления данных, разработан FragmentDriver для отображения списка гонщиков с использованием RecyclerView и адаптера DriversAdapter, реализована обработка долгого нажатия на элементы списка с выводом диалогового окна, содержащего опции просмотра, обновления и удаления гонщика. Созданы отдельные Activity для добавления и редактирования данных AddDriverActivity и EditDriverActivity с формами для ввода и изменения информации о гонщиках.

Третий этап включал интеграцию с облачной базой данных Supabase и реализацию синхронизации данных. Настроено подключение к Supabase через REST API с использованием Retrofit, созданы интерфейсы SupabaseApi и классы SupabaseDriver и SupabaseService для работы с удаленной базой данных. Реализована двусторонняя синхронизация данных между локальной базой Room и облачной базой Supabase, включая автоматическую загрузку данных при запуске приложения, синхронизацию операций добавления, обновления и удаления в фоновом режиме, а также функцию ручного обновления данных через кнопку обновления. Добавлена обработка ошибок

сети и отображение времени последней загрузки данных для информирования пользователя.

Четвертый этап был направлен на доработку функциональности и оптимизацию работы приложения. Реализована обработка JSON данных для получения информации об автомобилях гонщиков через внешний API с использованием `HttpURLConnection` и выполнения запросов в отдельном потоке `Thread` для обеспечения отзывчивости интерфейса. Добавлены дополнительные фрагменты `FragmentCar` и `FragmentAccount` для отображения карт и информации об аккаунте, улучшена обработка ошибок и добавлено логирование операций для отладки. Проведено тестирование всех функций приложения, включая проверку работы базы данных, синхронизации с Supabase, корректности отображения данных и обработки различных сценариев использования приложения.

3. Результат работы программы

В качестве результата работы программы представлено пару фотографий интерфейса. На рисунках 1-2 показан пример работы программы.



Рисунок 1 – Выбор гонщика

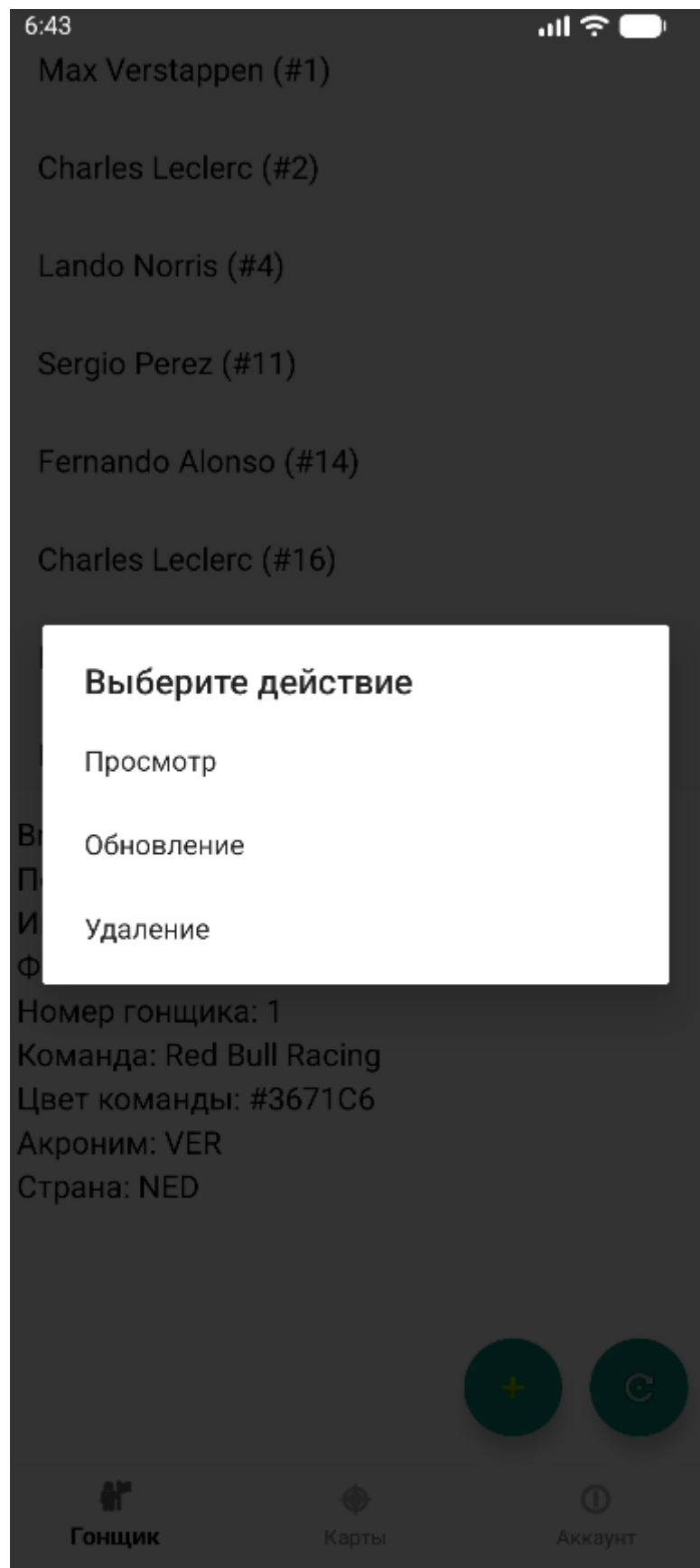


Рисунок 2 – Просмотр действия для гонщика

4. Вывод

В ходе выполнения лабораторной работы разработано мобильное приложение для Android на Kotlin, реализующее работу с базой данных и интеграцию с облачными сервисами. Приложение позволяет управлять информацией о гонщиках Формулы 1, включая просмотр списка, добавление новых записей, редактирование существующих данных и удаление записей. Использована библиотека Room для организации локальной базы данных, что обеспечивает надежное хранение и быстрый доступ к данным. Реализована синхронизация с облачной базой данных Supabase, что позволяет работать с актуальными данными и обеспечивает возможность использования приложения на разных устройствах. Интерфейс приложения построен с использованием фрагментов и RecyclerView, что обеспечивает удобную навигацию и эффективное отображение больших объемов данных.

Выполнение работы позволило освоить ключевые технологии разработки мобильных приложений под Android, включая работу с базами данных через Room, создание пользовательского интерфейса с использованием фрагментов и RecyclerView, обработку сетевых запросов через Retrofit, парсинг JSON данных и выполнение асинхронных операций с использованием Coroutines и Thread. Приложение демонстрирует практическое применение современных подходов к разработке мобильных приложений, включая архитектурные паттерны, работу с облачными сервисами и обеспечение отзывчивости пользовательского интерфейса. Разработанное решение может быть расширено дополнительным функционалом, таким как карты для визуализации данных и система регистрации пользователей, что открывает возможности для дальнейшего развития проекта.

5. Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:
#mainactivity.kt

```
package com.example.autotasks

import android.content.Intent
import android.os.Bundle
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.lifecycleScope
import androidx.fragment.app.commit
import
com.google.android.material.bottomnavigation.BottomNavigationView
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext
import kotlinx.coroutines.runBlocking
import kotlinx.coroutines.delay
import org.json.JSONArray
import java.net.HttpURLConnection
import java.net.URL
import android.net.ConnectivityManager
import android.net.NetworkInfo
import android.widget.Toast
import com.example.autotasks.database.AppDatabase
import com.example.autotasks.database.Driver
import com.example.autotasks.network.SupabaseService
import com.example.autotasks.network.SupabaseSync
```

```

import
com.google.android.material.floatingactionbutton.FloatingActionButton

@Suppress("DEPRECATION")
class MainActivity : AppCompatActivity() {

    val drivers = mutableListOf<Driver>()
    var currentDriverIndex = 0
    var lastLoadTime: Long = 0 // Время последней загрузки данных

    private lateinit var fragmentDriver: FragmentDriver
    private lateinit var database: AppDatabase

    companion object {
        const val REQUEST_ADD_DRIVER = 1
        const val REQUEST_EDIT_DRIVER = 2
    }

    private fun isNetworkAvailable(): Boolean {
        val connectivityManager =
getSystemService(CONNECTIVITY_SERVICE) as ConnectivityManager
        val networkInfo: NetworkInfo? =
connectivityManager.activeNetworkInfo
        return networkInfo?.isConnected == true
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }

```

```

        database = AppDatabase.getDatabase(this)

        if (!isNetworkAvailable()) {
            Toast.makeText(this, "Нет интернета",
                Toast.LENGTH_SHORT).show()
        }

        val bottomNav =
            findViewById<BottomNavigationView>(R.id.bottomNavigation)
        val fabAdd = findViewById<FloatingActionButton>(R.id.fabAdd)
        val fabRefresh =
            findViewById<FloatingActionButton>(R.id.fabRefresh)

        lifecycleScope.launch {
            // Сначала загружаем из локальной БД для быстрого отображения
            loadDriversFromDatabase()

            // Убеждаемся, что lastLoadTime установлен
            if (lastLoadTime == 0L) {
                lastLoadTime = System.currentTimeMillis()
                android.util.Log.w("MainActivity", "lastLoadTime не был
установлен, устанавливаем вручную")
            }

            // Создаем фрагмент гонщиков сразу, чтобы UI не был пустым
            fragmentDriver = FragmentDriver.newInstance(
                drivers,
                ::loadDriverInfo,
                ::showDriverActionsDialog,
                ::refreshDrivers,

```

```

        { lastLoadTime } // Функция для получения времени последней
загрузки
    )
    supportFragmentManager.commit {
        replace(R.id.fragmentContainer, fragmentDriver)
    }

    // Обновляем фрагмент после создания, чтобы показать время
загрузки
    kotlinx.coroutines.delay(100) // Даем время adapter
инициализироваться
    if (::fragmentDriver.isInitialized) {
        fragmentDriver.refreshDrivers()
    }
}

// Синхронизация с Supabase при запуске - загружаем актуальные
данные из реальной БД
Thread {
    try {
        // Небольшая задержка, чтобы UI успел отобразиться
        Thread.sleep(500)

        if (isNetworkAvailable()) {
            android.util.Log.d("MainActivity", "Начинаем синхронизацию
с Supabase...")
            runBlocking {
                try {
                    // Загружаем актуальные данные из Supabase
                    loadDriversFromSupabaseSync()

```

```

        android.util.Log.d("MainActivity", "Синхронизация с
Supabase завершена")

        // Перегружаем из локальной БД (которая теперь
обновлена)

        loadDriversFromDatabase()

        // Обновляем UI
        withContext(Dispatchers.Main) {
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
        }
    } catch (e: Exception) {
        android.util.Log.e("MainActivity", "Ошибка
синхронизации с Supabase: ${e.message}")
        e.printStackTrace()
        withContext(Dispatchers.Main) {
            // В случае ошибки просто загружаем из локальной
БД

            lifecycleScope.launch {
                loadDriversFromDatabase()
                if (::fragmentDriver.isInitialized) {
                    fragmentDriver.refreshDrivers()
                }
            }
        }
    }
} else {

```

```

        android.util.Log.w("MainActivity", "Нет интернета,
используем только локальную БД")
    }
} catch (e: Exception) {
    android.util.Log.e("MainActivity", "Ошибка в потоке
синхронизации: ${e.message}")
    e.printStackTrace()
}
}.start()

```

```

fabAdd.setOnClickListener {
    val intent = Intent(this, AddDriverActivity::class.java)
    startActivityForResult(intent, REQUEST_ADD_DRIVER)
}

```

```

fabRefresh.setOnClickListener {
    // Обновляем данные из Supabase и обновляем список
    refreshFromSupabase()
}

```

```

bottomNav.setOnItemSelectedListener { item ->
    when (item.itemId) {
        R.id.menu_driver -> {
            // фрагмент гонщиков
            supportFragmentManager.commit {
                replace(R.id.fragmentContainer, fragmentDriver)
            }
            true
        }
        R.id.menu_cards -> {

```

```

        // Показываем фрагмент карт
        supportFragmentManager.commit {
            replace(
                R.id.fragmentContainer,
                FragmentCar.newInstance(::loadCarInfo,
currentDriverIndex, "")
            )
        }
        true
    }
    R.id.menu_account -> {
        // Показываем фрагмент аккаунта
        supportFragmentManager.commit {
            replace(R.id.fragmentContainer,
FragmentAccount.newInstance())
        }
        true
    }
    else -> false
}
}
}

```

```

// Загружаем гонщиков из БД
private suspend fun loadDriversFromDatabase() {
    withContext(Dispatchers.IO) {
        try {

```



```

        // Добавляем небольшую задержку для гарантии реальной
загрузки из БД
        kotlinx.coroutines.delay(100)

        // Сначала проверяем количество записей в БД
        val count = database.driverDao().getDriversCount()
        android.util.Log.d("MainActivity", "Количество записей в БД
(COUNT): $count")

        // Затем загружаем все записи - принудительно, без кэша
        val dbDrivers = database.driverDao().getAllDrivers()

        android.util.Log.d("MainActivity", "Загружено из БД:
${dbDrivers.size} гонщиков")
        android.util.Log.d("MainActivity", "Время загрузки:
${System.currentTimeMillis()}")

        // Проверяем, что количество совпадает
        if (count != dbDrivers.size) {
            android.util.Log.e("MainActivity", "ОШИБКА:
Несоответствие! COUNT=$count, getAllDrivers=${dbDrivers.size}")
        }

        // Логируем детали загруженных гонщиков
        if (dbDrivers.isNotEmpty()) {
            android.util.Log.d("MainActivity", "ID загруженных
гонщиков: ${dbDrivers.map { it.id }}" )
            android.util.Log.d("MainActivity", "Номера гонщиков:
${dbDrivers.map { it.driverNumber }}" )
        } else {

```

```
        android.util.Log.w("MainActivity", "ВНИМАНИЕ: БД  
вернула пустой список!")  
    }
```

```
    withContext(Dispatchers.Main) {  
        val oldSize = drivers.size  
        val oldIds = drivers.map { it.id }.toSet()  
  
        // Полностью очищаем список перед добавлением новых  
        данных
```

```
        drivers.clear()
```

```
        // Создаем новый список из загруженных данных  
        val newDriversList = dbDrivers.toMutableList()  
        drivers.addAll(newDriversList)
```

```
        val newIds = drivers.map { it.id }.toSet()
```

```
        // Обновляем время последней загрузки  
        lastLoadTime = System.currentTimeMillis()
```

```
        android.util.Log.d("MainActivity", "Список обновлен. Старый  
размер: $oldSize, Новый размер: ${drivers.size}")
```

```
        // Логируем изменения  
        if (oldIds != newIds) {  
            val removed = oldIds - newIds  
            val added = newIds - oldIds  
            if (removed.isNotEmpty()) {
```

```

        android.util.Log.d("MainActivity", "Удалены ID:
$removed")
    }
    if (added.isNotEmpty()) {
        android.util.Log.d("MainActivity", "Добавлены ID:
$dadded")
    }
}

// Если список стал пустым, но был не пустым - это проблема
if (oldSize > 0 && drivers.isEmpty()) {
    android.util.Log.w("MainActivity", "ВНИМАНИЕ: Список
стал пустым после загрузки из БД! Старый размер: $oldSize")
}

// Если размер не совпадает с БД - это проблема
if (drivers.size != dbDrivers.size) {
    android.util.Log.e("MainActivity", "ОШИБКА: Размер
списка не совпадает с БД! Список: ${drivers.size}, БД: ${dbDrivers.size}")
}
}
} catch (e: Exception) {
    e.printStackTrace()
    android.util.Log.e("MainActivity", "Ошибка загрузки из БД:
${e.message}")
    // В случае ошибки не очищаем список
    withContext(Dispatchers.Main) {
        android.util.Log.w("MainActivity", "Ошибка загрузки,
сохраняем текущий список (размер: ${drivers.size})")
    }
}

```

```

        }
    }
}

// Обновление данных из Supabase (для кнопки refresh)
private fun refreshFromSupabase() {
    if (!isNetworkAvailable()) {
        Toast.makeText(this, "Нет интернета",
            Toast.LENGTH_SHORT).show()
        return
    }

    // Показываем индикатор загрузки
    Toast.makeText(this, "Обновление данных...",
        Toast.LENGTH_SHORT).show()

    Thread {
        try {
            runBlocking {
                android.util.Log.d("MainActivity", "Начало обновления из
Supabase...")

                // Загружаем актуальные данные из Supabase
                loadDriversFromSupabaseSync()
                android.util.Log.d("MainActivity", "Синхронизация с
Supabase завершена")

                // Перезагружаем из локальной БД (которая теперь
обновлена)
                loadDriversFromDatabase()
            }
        }
    }
}

```

```

        // Обновляем UI
        withContext(Dispatchers.Main) {
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
            Toast.makeText(this@MainActivity, "Данные обновлены",
Toast.LENGTH_SHORT).show()
        }
    }
} catch (e: Exception) {
    android.util.Log.e("MainActivity", "Ошибка обновления из
Supabase: ${e.message}")
    e.printStackTrace()
    runOnUiThread {
        Toast.makeText(this@MainActivity, "Ошибка обновления:
${e.message}", Toast.LENGTH_SHORT).show()

        // В случае ошибки просто загружаем из локальной БД
        lifecycleScope.launch {
            loadDriversFromDatabase()
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
        }
    }
}
}.start()
}

```

// Загружаем гонциков из Supabase синхронно (для первого запуска)

```

private suspend fun loadDriversFromSupabaseSync() {
    withContext(Dispatchers.IO) {
        try {
            val response = SupabaseService.api.getAllDrivers(
                SupabaseService.API_KEY,
                SupabaseService.getAuthorizationHeader()
            )

            if (response.isSuccessful && response.body() != null) {
                val supabaseDrivers = response.body()!!

                // Сохраняем в БД с проверкой дубликатов
                supabaseDrivers.forEach { supDriver ->
                    // Проверяем, существует ли уже гонщик с таким номером
                    val existing =
                        database.driverDao().getDriverByNumber(supDriver.driverNumber)

                    if (existing == null) {
                        // Если нет - добавляем
                        val driver = Driver(
                            fullName = supDriver.fullName,
                            driverNumber = supDriver.driverNumber,
                            firstName = supDriver.firstName,
                            lastName = supDriver.lastName,
                            teamName = supDriver.teamName,
                            teamColour = supDriver.teamColour,
                            nameAcronym = supDriver.nameAcronym,
                            countryCode = supDriver.countryCode,
                            broadcastName = supDriver.broadcastName
                        )
                    }
                }
            }
        }
    }
}

```

```

        database.driverDao().insertDriver(driver)
    } else {
        // Если есть - обновляем
        val driver = Driver(
            id = existing.id,
            fullName = supDriver.fullName,
            driverNumber = supDriver.driverNumber,
            firstName = supDriver.firstName,
            lastName = supDriver.lastName,
            teamName = supDriver.teamName,
            teamColour = supDriver.teamColour,
            nameAcronym = supDriver.nameAcronym,
            countryCode = supDriver.countryCode,
            broadcastName = supDriver.broadcastName
        )
        database.driverDao().updateDriver(driver)
    }
}

// Не обновляем список здесь - это будет сделано после
синхронизации
}
} catch (e: Exception) {
    e.printStackTrace()
}
}
}

// Загружаем гонщиков из Supabase в отдельном потоке (для фоновой
синхронизации)
private fun loadDriversFromSupabase() {

```

```

Thread {
    try {
        val response = runBlocking {
            SupabaseService.api.getAllDrivers(
                SupabaseService.API_KEY,
                SupabaseService.getAuthorizationHeader()
            )
        }

        if (response.isSuccessful && response.body() != null) {
            val supabaseDrivers = response.body()!!

            // Сохраняем в БД с проверкой дубликатов
            runBlocking {
                supabaseDrivers.forEach { supDriver ->
                    val existing =
                        database.driverDao().getDriverByNumber(supDriver.driverNumber)

                    if (existing == null) {
                        val driver = Driver(
                            fullName = supDriver.fullName,
                            driverNumber = supDriver.driverNumber,
                            firstName = supDriver.firstName,
                            lastName = supDriver.lastName,
                            teamName = supDriver.teamName,
                            teamColour = supDriver.teamColour,
                            nameAcronym = supDriver.nameAcronym,
                            countryCode = supDriver.countryCode,
                            broadcastName = supDriver.broadcastName
                        )
                    }
                }
            }
        }
    }
}

```



```

        database.driverDao().insertDriver(driver)
    } else {
        val driver = Driver(
            id = existing.id,
            fullName = supDriver.fullName,
            driverNumber = supDriver.driverNumber,
            firstName = supDriver.firstName,
            lastName = supDriver.lastName,
            teamName = supDriver.teamName,
            teamColour = supDriver.teamColour,
            nameAcronym = supDriver.nameAcronym,
            countryCode = supDriver.countryCode,
            broadcastName = supDriver.broadcastName
        )
        database.driverDao().updateDriver(driver)
    }
}

// Обновляем список
runOnUiThread {
    lifecycleScope.launch {
        loadDriversFromDatabase()
        if (::fragmentDriver.isInitialized) {
            fragmentDriver.refreshDrivers()
        }
    }
}

} catch (e: Exception) {

```

```

        e.printStackTrace()
    }
}.start()
}

private fun refreshDrivers() {
    lifecycleScope.launch {
        try {
            // Загружаем данные из БД
            loadDriversFromDatabase()

            // Убеждаемся, что данные загружены перед обновлением
фрагмента

            // Небольшая задержка для гарантии, что данные в списке
обновлены

            kotlinx.coroutines.delay(50)

            // Обновляем фрагмент после загрузки данных
            if (::fragmentDriver.isInitialized && drivers.isNotEmpty()) {
                fragmentDriver.refreshDrivers()
            } else if (::fragmentDriver.isInitialized) {
                // Даже если список пустой, обновляем фрагмент
                fragmentDriver.refreshDrivers()
            }
        } catch (e: Exception) {
            e.printStackTrace()
            // В случае ошибки все равно пытаемся обновить фрагмент
            if (::fragmentDriver.isInitialized) {
                fragmentDriver.refreshDrivers()
            }
        }
    }
}

```

```

    }
}

// Диалог выбора действий
private fun showDriverActionsDialog(driver: Driver) {
    val options = arrayOf("Просмотр", "Обновление", "Удаление")

    AlertDialog.Builder(this)
        .setTitle("Выберите действие")
        .setItems(options) { _, which ->
            when (which) {
                0 -> showDriverDetails(driver)
                1 -> editDriver(driver)
                2 -> confirmDeleteDriver(driver)
            }
        }
        .show()
}

private fun showDriverDetails(driver: Driver) {
    val details = ""
        Полное имя: ${driver.fullName}
        Имя: ${driver.firstName}
        Фамилия: ${driver.lastName}
        Номер гонщика: ${driver.driverNumber}
        Команда: ${driver.teamName}
        Цвет команды: #${driver.teamColour}
        Акроним: ${driver.nameAcronym}
        Страна: ${driver.countryCode}

```

```

        Broadcast Name: ${driver.broadcastName}
    """".trimIndent()

    AlertDialog.Builder(this)
        .setTitle("Информация о гонщике")
        .setMessage(details)
        .setPositiveButton("OK", null)
        .show()
}

private fun editDriver(driver: Driver) {
    val intent = Intent(this, EditDriverActivity::class.java)
    intent.putExtra("DRIVER_ID", driver.id)
    startActivityForResult(intent, REQUEST_EDIT_DRIVER)
}

private fun confirmDeleteDriver(driver: Driver) {
    AlertDialog.Builder(this)
        .setTitle("Удаление гонщика")
        .setMessage("Вы уверены, что хотите удалить  
${driver.fullName}?")
        .setPositiveButton("Да") { _, _ ->
            deleteDriver(driver)
        }
        .setNegativeButton("Нет", null)
        .show()
}

private fun deleteDriver(driver: Driver) {
    lifecycleScope.launch {

```

```

try {
    val driverNumber = driver.driverNumber
    val driverId = driver.id

    // Удаляем из БД
    withContext(Dispatchers.IO) {
        // Проверяем количество ДО удаления
        val countBefore = database.driverDao().getDriversCount()
        android.util.Log.d("MainActivity", "Удаление: количество ДО
удаления: $countBefore")

        database.driverDao().deleteDriver(driver)

        // Проверяем количество ПОСЛЕ удаления
        val countAfter = database.driverDao().getDriversCount()
        android.util.Log.d("MainActivity", "Удаление: количество
ПОСЛЕ удаления: $countAfter")

        // Проверяем, что действительно удалилось
        val deleted = database.driverDao().getDriverById(driverId)
        if (deleted != null) {
            android.util.Log.e("MainActivity", "ОШИБКА: Гонщик не
удален из БД! ID: $driverId")
        } else {
            android.util.Log.d("MainActivity", "Гонщик успешно
удален из БД. ID: $driverId, Number: $driverNumber")
        }

        // Проверяем, что количество уменьшилось
        if (countAfter >= countBefore) {

```

```

        android.util.Log.e("MainActivity", "ОШИБКА: Количество
не уменьшилось! Было: $countBefore, Стало: $countAfter")
    }
}

// Синхронизируем удаление с Supabase
SupabaseSync.syncDeleteDriver(driverNumber)

// Принудительно перезагружаем список из БД
loadDriversFromDatabase()

// Удаляем из локального списка на всякий случай
drivers.removeAll { it.id == driverId }

Toast.makeText(this@MainActivity, "Гонщик удален",
Toast.LENGTH_SHORT).show()

// Обновляем фрагмент
if (::fragmentDriver.isInitialized) {
    fragmentDriver.refreshDrivers()
}
} catch (e: Exception) {
    e.printStackTrace()
    Toast.makeText(this@MainActivity, "Ошибка удаления:
${e.message}", Toast.LENGTH_SHORT).show()
}
}
}

```

```

override fun onActivityResult(requestCode: Int, resultCode: Int, data:
Intent?) {
    super.onActivityResult(requestCode, resultCode, data)
    if (resultCode == RESULT_OK) {
        when (requestCode) {
            REQUEST_ADD_DRIVER, REQUEST_EDIT_DRIVER -> {
                // Сразу обновляем список - данные уже должны быть
сохранены в EditDriverActivity
                refreshDrivers()
            }
        }
    }
}

// Получаем информацию о выбранном гонщике из БД
private suspend fun loadDriverInfo(driver: Driver): String {
    return """
        Broadcast Name: ${driver.broadcastName}
        Полное имя: ${driver.fullName}
        Имя: ${driver.firstName}
        Фамилия: ${driver.lastName}
        Номер гонщика: ${driver.driverNumber}
        Команда: ${driver.teamName}
        Цвет команды: #${driver.teamColour}
        Акроним: ${driver.nameAcronym}
        Страна: ${driver.countryCode}
    """.trimIndent()
}

// Получаем данные автомобиля по номеру гонщика (из старого API)
private suspend fun loadCarInfo(driverNumber: Int): String {

```

```

return withContext(Dispatchers.IO) {
    try {
        val url =
            URL("https://api.openf1.org/v1/car_data?driver_number=$driverNumber&session
            _key=9159")

        val conn = url.openConnection() as HttpURLConnection
        conn.requestMethod = "GET"
        conn.connectTimeout = 5000
        conn.readTimeout = 5000

        val code = conn.responseCode
        if (code != 200) return@withContext "Ошибка запроса: $code"

        val data = conn.inputStream.bufferedReader().readText()
        val json = JSONArray(data)
        if (json.length() == 0) return@withContext "Данные автомобиля
        не найдены"

        val obj = json.getJSONObject(0)
        val speed = obj.optInt("speed")
        val nGear = obj.optInt("n_gear")
        val drs = obj.optInt("drs")
        val throttle = obj.optInt("throttle")
        val brake = obj.optInt("brake")
        val rpm = obj.optInt("rpm")
        val date = obj.optString("date")

        return@withContext """"
        Дата: $date
    }
}

```


Скорость: \$speed км/ч

Передача: \$nGear

DRS: \$drs

Газ: \$throttle%

Тормоз: \$brake

RPM: \$rpm

"".trimIndent()

} catch (e: Exception) {

return@withContext "Ошибка: \${e.message}"

}

}

}

}